

## Transferleistung Theorie/Praxis 6

|                               |                                                           |
|-------------------------------|-----------------------------------------------------------|
| <b>Matrikelnummer:</b>        | 12315                                                     |
| <b>Freigegebenes Thema:</b>   | Agile Dokumentation: Wie viel Dokumentation ist sinnvoll? |
| <b>Studiengang, Zenturie:</b> | Angewandte Informatik, A22a                               |

# Inhaltsverzeichnis

|                                                                                  |            |
|----------------------------------------------------------------------------------|------------|
| <b>Abbildungsverzeichnis .....</b>                                               | <b>II</b>  |
| <b>Tabellenverzeichnis .....</b>                                                 | <b>III</b> |
| <b>Quellcodeverzeichnis .....</b>                                                | <b>IV</b>  |
| <b>Abkürzungen .....</b>                                                         | <b>V</b>   |
| <b>1 Einleitung .....</b>                                                        | <b>1</b>   |
| 1.1 Problemstellung .....                                                        | 1          |
| 1.2 Zielsetzung und Forschungsfrage .....                                        | 1          |
| <b>2 Theoretische Grundlagen von Dokumentation<br/>in agilen Projekten .....</b> | <b>3</b>   |
| 2.1 Grundlagen agiler Softwareentwicklung .....                                  | 3          |
| 2.2 Merkmale guter Projektdokumentation .....                                    | 4          |
| 2.3 Vorteile guter Projektdokumentation .....                                    | 5          |
| 2.4 Die Rolle von Dokumentationsaufwand in agilen Projekten .....                | 6          |
| <b>3 Bestimmung eines sinnvollen<br/>Dokumentationsumfang .....</b>              | <b>8</b>   |
| 3.1 Bewertungskriterien eines sinnvollen Umfangs .....                           | 8          |
| 3.1.1 Zeitlicher und organisatorischer Umgang mit Dokumentation .....            | 8          |
| <b>4 Praxisbezug .....</b>                                                       | <b>10</b>  |
| 4.1 Projektbeschreibung von Finrep .....                                         | 10         |
| 4.2 Bewertung bestehender Dokumentation und<br>Dokumentationspraktiken .....     | 10         |
| 4.3 Handlungsempfehlung .....                                                    | 11         |
| <b>5 Fazit .....</b>                                                             | <b>13</b>  |
| 5.1 Beantwortung der Forschungsfrage .....                                       | 13         |
| 5.2 Ausblick .....                                                               | 13         |
| <b>Bibliographie .....</b>                                                       | <b>VI</b>  |

# Abbildungsverzeichnis

# Tabellenverzeichnis

# Quellcodeverzeichnis

# Abkürzungen

# 1 Einleitung

## 1.1 Problemstellung

In agilen Softwareprojekten steht die schnelle und flexible Entwicklung funktionsfähiger Software im Mittelpunkt. Anforderungen ändern sich häufig, Entwicklungszyklen sind kurz und Teams reagieren kontinuierlich auf neue fachliche oder technische Rahmenbedingungen. In diesem Umfeld wird Dokumentation häufig als Hindernis wahrgenommen. Einerseits ist Dokumentation notwendig, um Wissen nachvollziehbar festzuhalten, die Einarbeitung neuer Mitarbeitender zu erleichtern und langfristige Wartbarkeit sicherzustellen. Andererseits kann ein zu hoher Dokumentationsaufwand Entwicklungszeit binden und dadurch den agilen Grundgedanken einer schnellen, iterativen Produktentwicklung beeinträchtigen.

Dieser Konflikt zeigt sich insbesondere in laufenden Softwareprojekten, in denen bestehende Dokumentation über längere Zeit gewachsen ist oder unter Zeitdruck nur unvollständig gepflegt wurde. Wenn Dokumentation veraltet, unübersichtlich oder lückenhaft ist, kann dies zu erhöhtem Abstimmungsaufwand, Wissensverlust und Fehlern bei der Weiterentwicklung führen. Gleichzeitig ist es in agilen Projekten kaum sinnvoll, jede technische oder fachliche Entscheidung umfassend zu dokumentieren, da sich Inhalte durch neue Anforderungen schnell verändern können.

Daraus ergibt sich die zentrale Herausforderung, einen angemessenen Dokumentationsumfang zu bestimmen. Dokumentation muss ausreichend Informationen bereitstellen, um Projektwissen zu sichern und die Entwicklung zu unterstützen. Gleichzeitig darf sie nicht zu einem Selbstzweck werden oder den Entwicklungsfortschritt unverhältnismäßig verlangsamen. Vor diesem Hintergrund stellt sich die Frage, wie viel Dokumentation in agilen Projekten sinnvoll ist und nach welchen Kriterien entschieden werden kann, welche Inhalte dokumentiert werden sollten.

## 1.2 Zielsetzung und Forschungsfrage

Ziel dieser Transferleistung ist es, den sinnvollen Umfang von Dokumentation in agilen Softwareprojekten theoretisch zu untersuchen und auf ein praktisches Beispiel anzuwenden. Dabei soll herausgearbeitet werden, welche Merkmale gute Projektdokumentation auszeichnen, welchen Nutzen sie für Entwicklungsteams bietet und wie Dokumentationsaufwand in agilen Arbeitsweisen angemessen eingeordnet werden kann.

Der Praxisbezug erfolgt anhand des agilen Softwareprojekts Finrep. Dabei wird betrachtet, in welchem Zustand sich die bestehende Dokumentation befindet und welche Dokumentationspraktiken im Projekt erkennbar sind. Auf Basis der theoretischen Grundlagen sollen anschließend Bewertungskriterien abgeleitet werden, mit denen der bestehende Dokumentationsumfang eingeschätzt werden kann. Daraus werden Handlungsempfehlungen entwickelt, die zu einer besseren und gleichzeitig angemessenen Dokumentation beitragen sollen.

Die zentrale Forschungsfrage dieser Transferleistung lautet daher:

Wie viel Dokumentation ist in agilen Softwareprojekten sinnvoll, um Projektwissen nachhaltig zu sichern, ohne den Entwicklungsfortschritt unverhältnismäßig zu beeinträchtigen?

Zur Beantwortung dieser Forschungsfrage werden sowohl theoretische Grundlagen agiler Softwareentwicklung und Projektdokumentation als auch praktische Beobachtungen aus dem Projekt Finrep herangezogen. Der Fokus liegt dabei nicht auf einer möglichst umfangreichen Dokumentation, sondern auf der Frage, welche Dokumentation tatsächlich hilfreich, aktuell und nutzbar ist.



## 2 Theoretische Grundlagen von Dokumentation in agilen Projekten

### 2.1 Grundlagen agiler Softwareentwicklung

Agile Softwareentwicklung bezeichnet einen Ansatz der Softwareentwicklung, der auf kurze Entwicklungszyklen, frühes Feedback, enge Zusammenarbeit mit Stakeholdern und eine hohe Anpassungsfähigkeit gegenüber veränderten Anforderungen ausgerichtet ist. Im Gegensatz zu stark planungsgetriebenen Vorgehensmodellen wird der Entwicklungsprozess nicht primär als linearer Ablauf verstanden, bei dem Anforderungen, Entwurf, Implementierung und Test strikt nacheinander erfolgen. Vielmehr werden Softwareprodukte iterativ und inkrementell entwickelt, sodass lauffähige Teilergebnisse früh entstehen und regelmäßig überprüft werden können. Wissenschaftliche Übersichtsarbeiten beschreiben agile Methoden als leichtgewichtige Vorgehensweisen, die insbesondere auf dynamische Projektumfelder reagieren sollen, in denen Anforderungen nicht vollständig zu Beginn feststehen [1], [2].

Ein zentrales Merkmal agiler Entwicklung ist der Umgang mit Unsicherheit. Softwareprojekte sind häufig dadurch gekennzeichnet, dass fachliche Anforderungen, technische Rahmenbedingungen und organisatorische Prioritäten während des Projektverlaufs verändert werden. Agilität bedeutet daher nicht nur Schnelligkeit, sondern vor allem die Fähigkeit, Veränderungen wahrzunehmen, angemessen darauf zu reagieren und daraus Nutzen für das Projekt zu erzeugen [3]. Agile Entwicklung ist somit nicht mit ungeplantem oder undiszipliniertem Arbeiten gleichzusetzen. Stattdessen wird Veränderbarkeit systematisch in den Entwicklungsprozess eingebaut, beispielsweise durch kurze Iterationen, regelmäßige Reviews, Retrospektiven und eine enge Abstimmung zwischen Entwicklungsteam und Fachseite.

Agile Projekte beruhen außerdem stark auf Kommunikation und Zusammenarbeit. Da Entscheidungen häufig dezentral im Team getroffen werden, gewinnen gemeinsame Zielbilder, geteiltes Fachwissen und ein funktionierender Informationsfluss eine hohe Bedeutung. Empirische Untersuchungen zu agilen Teams zeigen, dass Teamorientierung, Koordination und ein gemeinsames Verständnis wesentliche Faktoren für die Wirksamkeit agiler Projektarbeit sind [4]. Gleichzeitig können Spezialisierung, unklare Zuständigkeiten und fehlende Abstimmung die Teamleistung beeinträchtigen. Daraus ergibt sich, dass agile Softwareentwicklung zwar weniger formale Übergaben als klassische Vorgehensmodelle vorsieht, aber keineswegs ohne strukturierte Informationsweitergabe auskommt.

Die Vorteile agiler Vorgehensweisen liegen insbesondere in der frühen Lieferung nutzbarer Software, der stärkeren Einbindung von Kunden und Anwendern sowie der Möglichkeit, Anforderungen während des Projektverlaufs zu präzisieren. Studien zeigen einen positiven Zusammenhang zwischen agilen Methoden und Projekterfolg, weisen jedoch zugleich darauf hin, dass dieser Erfolg von der konkreten Umsetzung und vom Projektkontext abhängt [4]. Agile Methoden sind damit kein

Selbstzweck, sondern müssen an Rahmenbedingungen wie Projektgröße, Kritikalität, Teamstruktur und organisatorisches Umfeld angepasst werden [5].

## 2.2 Merkmale guter Projektdokumentation

Projektdokumentation umfasst alle schriftlichen, grafischen oder digitalen Artefakte, die Informationen über Anforderungen, Architektur, fachliche Entscheidungen, Schnittstellen, Code, Tests, Betrieb oder organisatorische Abläufe festhalten. In Softwareprojekten dient Dokumentation nicht nur der Ablage von Informationen, sondern vor allem der Kommunikation zwischen verschiedenen Beteiligten. Dazu gehören Entwicklerinnen und Entwickler, Product Owner, Fachabteilungen, Tester, Betriebsteams, neue Teammitglieder sowie externe Stakeholder. Gute Dokumentation ist daher nicht allein daran zu messen, ob sie umfangreich ist, sondern ob sie für ihre Zielgruppe verständlich, auffindbar, aktuell und handlungsrelevant ist.

Ein wesentliches Qualitätsmerkmal guter Dokumentation ist Zweckorientierung. Dokumentation sollte einen klaren Nutzen erfüllen, beispielsweise die Erklärung einer Architekturentscheidung, die Beschreibung einer Schnittstelle, die Nachvollziehbarkeit einer fachlichen Anforderung oder die Unterstützung bei der Einarbeitung. Empirische Untersuchungen zur Nutzung von Softwaredokumentation zeigen, dass Dokumentation in der Praxis häufig nicht konsequent aktualisiert wird und deshalb nur dann langfristig nützlich bleibt, wenn sie einfach, relevant und mit vertretbarem Aufwand pflegbar ist [6]. Dokumentation sollte daher nicht möglichst vollständig im Sinne einer maximalen Informationssammlung sein, sondern so gestaltet werden, dass sie von den Beteiligten tatsächlich genutzt und fortgeschrieben wird.

Ein zweites Merkmal ist Aktualität. Veraltete Dokumentation kann in Softwareprojekten problematisch sein, weil sie falsche Annahmen erzeugt und dadurch Fehlentscheidungen begünstigt. Allerdings bedeutet Aktualität nicht, dass jedes Detail jederzeit vollständig dokumentiert sein muss. Vielmehr sollte vor allem jene Dokumentation aktuell gehalten werden, die für Entwicklung, Betrieb, Wartung oder Entscheidungen relevant ist. Die Nützlichkeit von Dokumentation hängt stark davon ab, ob die enthaltenen Informationen für die jeweilige Aufgabe relevant sind und ob geeignete Werkzeuge zur Erstellung und Pflege eingesetzt werden [7].

Ein drittes Qualitätsmerkmal ist Verständlichkeit. Dokumentation muss so formuliert sein, dass sie auch von Personen verstanden werden kann, die nicht an der ursprünglichen Entscheidung oder Implementierung beteiligt waren. Dies betrifft insbesondere neue Teammitglieder, Wartungsentwickler oder Personen aus angrenzenden Rollen. Verständlichkeit hängt dabei von klarer Sprache, konsistenter Struktur, geeigneter Granularität und nachvollziehbaren Beispielen ab. Qualitätsmodelle für Softwaredokumentation betonen deshalb neben fachlicher Korrektheit auch Struktur, Lesbarkeit und Nutzbarkeit der Dokumentation [8].

Ein weiteres Merkmal ist Nachvollziehbarkeit. In Softwareprojekten müssen Entscheidungen häufig später rekonstruiert werden können, etwa wenn technische Schulden bewertet, Fehler analysiert oder Architekturänderungen geplant werden. Gute Dokumentation hält daher nicht nur fest,

was entschieden wurde, sondern auch, warum eine Entscheidung getroffen wurde. Besonders bei fachlichen Anforderungen ist zudem die Verbindung zwischen Anforderung, Umsetzung und Test wichtig. Forschung zu agilem Requirements Engineering zeigt, dass agile Entwicklung und systematische Anforderungsarbeit nicht zwangsläufig im Widerspruch stehen. Anforderungen können in agilen Projekten beispielsweise durch User Stories, Backlogs, Akzeptanzkriterien und Testfälle leichtgewichtig dokumentiert werden [9], [10].

Gute Projektdokumentation ist zuletzt grundsätzlich wartbar. Eine Dokumentation, die nur mit hohem Aufwand gepflegt werden kann, wird in agilen Projekten schnell vernachlässigt, weil Entwicklungszeit knapp ist und der Fokus auf lauffähiger Software liegt. Wartbarkeit bedeutet deshalb, dass Dokumentation möglichst nah an den Arbeitsprozessen entsteht, klar verantwortet wird und nicht unnötig redundant ist. Studien zur Nutzung und Relevanz von Softwaredokumentation zeigen, dass Dokumentation vor allem dann hilfreich ist, wenn sie aufgabenbezogen, aktuell und mit angemessenem Aufwand pflegbar bleibt [6], [7]. Zudem ist insbesondere für die spätere Wartung entscheidend, dass Dokumentation Systemverhalten, Struktur und Änderungsfolgen verständlich unterstützt [11]. Wo Informationen bereits im Code, in Tests, im Backlog oder in automatisierten Systemen vorhanden sind, sollte Dokumentation diese Informationen daher nicht unreflektiert duplizieren, sondern ergänzen, einordnen und erklären. Gute Dokumentation ist somit zweckmäßig, aktuell, verständlich, nachvollziehbar und mit angemessenem Aufwand pflegbar.

## 2.3 Vorteile guter Projektdokumentation

Gute Projektdokumentation erfüllt in Softwareprojekten mehrere Funktionen und hat mehrere Vorteile.

Ein zentraler Vorteil guter Projektdokumentation liegt im Wissenstransfer innerhalb des Teams. Softwareentwicklung ist wissensintensive Arbeit, bei der viele relevante Informationen nicht nur im Code, sondern auch in fachlichen Entscheidungen, technischen Abwägungen und implizitem Erfahrungswissen enthalten sind. Wenn dieses Wissen ausschließlich in den Köpfen einzelner Personen vorhanden ist, entstehen Risiken bei Krankheit, Rollenwechsel, Fluktuation oder wachsender Teamgröße. Empirische Untersuchungen zeigen, dass Dokumentation in agilen Teams trotz direkter Kommunikation notwendig bleibt, um internes Wissen verfügbar zu halten und Informationsverluste zu vermeiden [12]. Dokumentation reduziert diese Abhängigkeit, indem zentrale Informationen dauerhaft festgehalten und für spätere Entwicklungs-, Wartungs- und Einarbeitungssituationen nutzbar bleiben [6].

Ein weiterer Vorteil besteht in der Unterstützung von Wartung und Weiterentwicklung bestehender Software. Gerade langlebige Softwaresysteme werden häufig von Personen bearbeitet und gewartet, die nicht an der ursprünglichen Entwicklung beteiligt waren. Untersuchungen zur Softwarewartung zeigen, dass bestimmte Dokumentationsartefakte besonders relevant sind, um Systemverhalten, Struktur und Änderungsfolgen zu verstehen [11]. Auch Studien zur praktischen Nutzung von Dokumentation zeigen, dass Softwareentwickelnde Dokumentation in unterschiedlichen Arbeitssituationen verwenden, etwa zur Orientierung, Analyse und Veränderung bestehender Systeme [6].

Darüber hinaus verbessert Dokumentation die Kommunikation zwischen Rollen und Stakeholdern. In agilen Projekten wird zwar häufig direkte Kommunikation bevorzugt, jedoch kann diese nicht alle Informationsbedarfe dauerhaft abdecken. Entscheidungen aus Meetings, fachliche Annahmen oder technische Einschränkungen können verloren gehen, wenn sie nicht festgehalten werden. Fallstudien zu großskaliger Softwareentwicklung zeigen, dass Kommunikationslücken zu Qualitätsproblemen, verfehlten Kundenerwartungen und unnötigem Aufwand führen können [13]. Dokumentation kann solche Lücken verringern, indem sie ein gemeinsames Referenzsystem schafft und zentrale Informationen für spätere Projektphasen verfügbar hält.

Gute Dokumentation unterstützt außerdem die Qualitätssicherung. Anforderungen, Akzeptanzkriterien, Testfälle und bekannte Einschränkungen schaffen eine Grundlage dafür, ob eine Umsetzung fachlich korrekt ist. Forschung zu agilem Requirements Engineering zeigt, dass informelle Kommunikation zwar Flexibilität ermöglicht, aber auch Risiken erzeugen kann, wenn Anforderungen unklar oder nur mündlich weitergegeben werden [14]. Systematische Literaturübersichten zeigen außerdem, dass agile Requirements-Engineering-Praktiken häufig auf kontinuierliche Priorisierung, enge Stakeholder-Kommunikation und leichtgewichtige Anforderungsartefakte setzen [15]. Dokumentation unterstützt diese Praktiken, wenn sie Anforderungen nicht unnötig verfestigt, sondern verständlich und überprüfbar macht.

Schließlich erhöht Dokumentation die organisatorische Nachvollziehbarkeit. Dies betrifft vor allem Projekte, die über längere Zeiträume laufen, mehrere Teams einbinden oder regulatorischen und betrieblichen Anforderungen unterliegen. Empirische Untersuchungen zu großskaliger agiler Systementwicklung zeigen, dass der Umgang mit Anforderungen und zugehörigem Wissen in größeren Organisationen eine besondere Herausforderung darstellt [16]. Je stärker ein Projekt wächst, desto weniger reicht rein informelle Kommunikation aus. Dokumentation wird dann zu einem Mittel, um Wissen über Teamgrenzen hinweg verfügbar zu machen und Abstimmungsprobleme zu reduzieren.

## **2.4 Die Rolle von Dokumentationsaufwand in agilen Projekten**

Der Dokumentationsaufwand nimmt in agilen Projekten eine ambivalente Rolle ein. Einerseits wird Dokumentation benötigt, um Wissen zu sichern, Anforderungen nachvollziehbar zu machen und langfristige Wartbarkeit zu ermöglichen. Andererseits verursacht jede Dokumentationsaktivität Aufwand, der nicht unmittelbar in neue Funktionalität fließt. Agile Projekte stehen daher vor der Aufgabe, ein angemessenes Verhältnis zwischen Dokumentation und Entwicklungsfortschritt zu finden. Dabei geht es nicht um die Frage, ob dokumentiert werden soll, sondern welche Informationen in welcher Tiefe, zu welchem Zeitpunkt und für welche Zielgruppe dokumentiert werden müssen.

Empirische Untersuchungen zu interner Dokumentation in agilen Teams zeigen, dass Dokumentation in agilen Projekten zugleich als notwendig und häufig vernachlässigt wahrgenommen wird [12]. Die stärkere Betonung direkter Kommunikation führt in der Praxis häufig dazu, dass Dokumentation reduziert wird. Dies kann in kleinen, stabilen und eng zusammenarbeitenden Teams zunächst funktionieren. Mit wachsender Systemkomplexität, längeren Projektlaufzeiten oder

personellen Veränderungen steigt jedoch der Bedarf an dokumentiertem Wissen. Dokumentationsaufwand muss daher kontextabhängig bewertet werden.

Ein häufiger Fehler besteht darin, agile Dokumentation mit möglichst wenig Dokumentation gleichzusetzen. Die wissenschaftliche Literatur legt jedoch nahe, dass nicht die Menge, sondern der Nutzen der Dokumentation entscheidend ist. Systematische Literaturübersichten zu Dokumentationspraktiken in agiler Softwareentwicklung zeigen, dass in der Praxis unterschiedliche Auffassungen darüber bestehen, was in agilen Projekten dokumentiert werden sollte [17]. Daraus folgt, dass Dokumentation in agilen Projekten bewusst gestaltet werden muss und nicht lediglich nebenbei entstehen kann.

Der sinnvolle Dokumentationsaufwand hängt von mehreren Einflussfaktoren ab. Dazu gehören die Größe des Teams, die Fluktuation, die fachliche und technische Komplexität des Systems, die Kritikalität der Software, die Anzahl externer Schnittstellen, regulatorische Anforderungen und die erwartete Lebensdauer des Produkts. In kleinen Projekten mit stabiler Teamzusammensetzung kann ein geringerer formaler Dokumentationsumfang ausreichend sein. In größeren oder langfristig betriebenen Systemen ist dagegen eine stärkere Dokumentation notwendig, weil mehr Personen auf dieselben Informationen zugreifen müssen und Entscheidungen über längere Zeit nachvollziehbar bleiben sollen. Zu viel Formalisierung kann die Anpassungsfähigkeit reduzieren, während zu wenig Struktur Risiken für Qualität, Koordination und Wartbarkeit erzeugt [5].

Dokumentation sollte in agilen Projekten möglichst inkrementell und arbeitsnah entstehen. Das bedeutet, dass Dokumentation nicht erst am Ende eines Projekts nachträglich erstellt wird, sondern parallel zur Entwicklung gepflegt wird. Besonders geeignet sind leichtgewichtige Artefakte wie User Stories, Akzeptanzkriterien, Architekturentscheidungen, Schnittstellenbeschreibungen, Testdokumentation, Betriebsanleitungen und kurze Onboarding-Hinweise. Diese Artefakte müssen nicht zwangsläufig umfangreich sein, sollten aber zentrale Informationen enthalten, die ohne Dokumentation schwer rekonstruierbar wären. Forschung zu agilem Requirements Engineering zeigt, dass leichtgewichtige und iterativ gepflegte Artefakte dazu beitragen können, Dokumentation und Flexibilität miteinander zu verbinden [10], [15].

Der Dokumentationsaufwand ist daher nicht isoliert zu betrachten, sondern als Investition in zukünftige Entwicklungsfähigkeit. Kurzfristig kann eine Reduzierung von Dokumentation Entwicklungszeit sparen. Langfristig kann fehlende Dokumentation jedoch zu höheren Kosten führen, etwa durch längere Einarbeitungszeiten, wiederholte Rückfragen, Fehlinterpretationen, technische Schulden oder erschwerte Wartung. Gute agile Dokumentation versucht, diese beiden Perspektiven auszubalancieren. Sie vermeidet unnötige Dokumente, hält aber diejenigen Informationen fest, die für Verständnis, Weiterentwicklung und Betrieb wesentlich sind.



## 3 Bestimmung eines sinnvollen Dokumentationsumfang

### 3.1 Bewertungskriterien eines sinnvollen Umfangs

Aus den theoretischen Grundlagen lassen sich mehrere Kriterien ableiten, mit denen der Umfang von Projektdokumentation bewertet werden kann.

Das erste Kriterium ist die Zweckmäßigkeit. Jede Dokumentation sollte einem konkreten Zweck dienen, etwa der Einarbeitung, Wartung, Fehleranalyse, Entscheidungsnachverfolgung oder dem Betrieb des Systems. Dokumente ohne klaren Verwendungszweck sind kritisch zu bewerten, da sie zusätzlichen Pflegeaufwand erzeugen, ohne den Entwicklungsprozess erkennbar zu unterstützen.

Ein zweites Kriterium ist die Relevanz der Inhalte. Dokumentiert werden sollten vor allem Informationen, die für das Verständnis des Systems notwendig sind und nicht ohne Weiteres aus Code, Tests oder Tickets hervorgehen. Dazu zählen insbesondere fachliche Besonderheiten, zentrale Architekturentscheidungen, externe Schnittstellen, Betriebsabläufe und Abhängigkeiten zu anderen Systemen.

Ein weiteres Kriterium ist die Aktualisierbarkeit. Der Umfang der Dokumentation ist nur dann angemessen, wenn das Team realistisch in der Lage ist, die Inhalte aktuell zu halten. Eine umfangreiche Dokumentation, die im Projektalltag nicht gepflegt werden kann, ist weniger sinnvoll als eine kleinere, aber verlässliche Dokumentationsbasis.

Zusätzlich ist die Auffindbarkeit und Verständlichkeit zu bewerten. Dokumentation erfüllt ihren Zweck nur, wenn relevante Informationen schnell gefunden und von der jeweiligen Zielgruppe verstanden werden können. Eine unübersichtliche Dokumentationsstruktur kann den Nutzen der Dokumentation auch dann verringern, wenn grundsätzlich viele Inhalte vorhanden sind.

Schließlich muss der Projektkontext berücksichtigt werden. Der notwendige Umfang steigt mit der Komplexität des Systems, der Anzahl beteiligter Personen, der Lebensdauer des Produkts, der Kritikalität einzelner Funktionen und der Bedeutung externer Schnittstellen. Ein sinnvoller Dokumentationsumfang ist daher immer kontextabhängig und muss an die tatsächlichen Risiken und Informationsbedarfe des Projekts angepasst werden.

Damit ergibt sich als Bewertungsmaßstab, dass Dokumentation nur dann sinnvoll umfangreich ist, wenn sie die für Entwicklung, Wartung und Betrieb wesentlichen Informationen bereitstellt, ohne durch unnötige oder schwer pflegbare Inhalte den agilen Entwicklungsprozess zu belasten.

#### 3.1.1 Zeitlicher und organisatorischer Umgang mit Dokumentation

Dokumentation sollte möglichst parallel zur Entwicklung entstehen und bei Änderungen unmittelbar angepasst werden. Leichtgewichtige und iterativ gepflegte Dokumentationsartefakte ermög-

lichen es, notwendige Informationen festzuhalten, ohne die Anpassungsfähigkeit des Projekts unnötig einzuschränken [10], [15].

Organisatorisch sollte die Dokumentationspflege als Bestandteil der jeweiligen Entwicklungsaufgabe behandelt werden. Da Dokumentation in agilen Teams häufig als notwendig angesehen, im Projektalltag jedoch vernachlässigt wird, sind klare Zuständigkeiten und regelmäßige Überprüfungen erforderlich [12].

Ein allgemeingültiger fester Zeitanteil für Dokumentation ist nicht sinnvoll. Der erforderliche Aufwand hängt unter anderem von der Komplexität, Kritikalität und Lebensdauer des Systems ab. Dokumentationsaufwand sollte daher bedarfsorientiert festgelegt werden, sodass ausreichende Nachvollziehbarkeit entsteht, ohne den Entwicklungsfortschritt unverhältnismäßig zu beeinträchtigen [5], [17].

## 4 Praxisbezug

### 4.1 Projektbeschreibung von Finrep

IDL Finrep ist eine Windows-Desktop-Anwendung zur Erstellung und Validierung regulatorischer Finanzmeldungen. Sie unterstützt Banken bei der Erfüllung ihrer Meldepflichten gegenüber europäischen Zentralbanken und verarbeitet hierfür verschiedene Berichtsstandards, insbesondere FINREP. Neben dem Import und der Zuordnung von Finanzdaten umfasst die Anwendung Validierungs-, Auswertungs- und Exportfunktionen. Aufgrund der langen Projektlaufzeit, der regulatorischen Anforderungen und der regelmäßigen Anpassung an neue Taxonomien handelt es sich um ein fachlich komplexes und kontinuierlich weiterentwickeltes Softwaresystem.

### 4.2 Bewertung bestehender Dokumentation und Dokumentationspraktiken

Die Dokumentationslandschaft von Finrep ist insgesamt umfangreich und setzt sich aus unterschiedlichen Artefakten zusammen. Im Repository befindet sich eine ausführliche README-Datei. Zusätzlich stehen in Confluence unter anderem eine Architekturübersicht, ein Benutzerhandbuch, ein Installationsleitfaden, eine FAQ-Sammlung, ein Glossar, Berechtigungsmatrizen sowie Dokumentationen zu Taxonomien und DPM 2.0 zur Verfügung. Ergänzt werden diese Inhalte durch Sprint-Reviews und strukturierte Jira-Vorgänge, in denen Fehler, Anforderungen, Tests und technische Aufgaben dokumentiert werden.

Hinsichtlich der Zweckmäßigkeit ist die Dokumentation überwiegend positiv zu bewerten. Die meisten Artefakte besitzen einen erkennbaren Verwendungszweck und richten sich an konkrete Zielgruppen. Benutzer- und Installationsleitfäden unterstützen Anwender und Betrieb, während Architektur- und Entwicklungsdokumentationen insbesondere für die Einarbeitung und Wartung relevant sind. Jira-Vorgänge halten Anforderungen, Fehlerbilder und Umsetzungsschritte fest und unterstützen dadurch die Abstimmung innerhalb des Entwicklungsteams. Einschränkend bestehen einzelne inhaltliche Überschneidungen, beispielsweise durch mehrfach vorhandene Feature-Matrizen. Daneben sind weiterhin veraltete oder nicht mehr verwendete Seiten vorhanden, auch wenn diese teilweise entsprechend gekennzeichnet wurden.

Die Relevanz der dokumentierten Inhalte ist ebenfalls grundsätzlich hoch. Insbesondere die Architekturübersicht und zahlreiche Jira-Vorgänge halten Informationen fest, die nicht unmittelbar aus dem Quellcode hervorgehen. Hierzu gehören fachliche Hintergründe, kundenspezifische Anforderungen, technische Einschränkungen und Begründungen für bestimmte Entscheidungen. Damit wird ein Teil des für die Weiterentwicklung wichtigen Kontextwissens dauerhaft verfügbar gemacht.

Defizite zeigen sich vor allem bei der Dokumentation innerhalb des Quellcodes. Zentrale Geschäftsobjekte enthalten teilweise keine erläuternden Kommentare. Auch Datenzugriffsmethoden mit



eingebetteten SQL-Anweisungen dokumentieren nicht immer ausreichend, welchem Zweck eine Abfrage dient und welche Tabellen oder fachlichen Prozesse davon betroffen sind. Bei komplexen Berechnungs- und Validierungsabläufen besteht dadurch eine starke Abhängigkeit vom Erfahrungswissen einzelner Teammitglieder. Der zeitweise Verlust von Detailwissen zur Einbindung einer Berechnungseingabe während der Taxonomieumstellung verdeutlicht dieses Risiko.

Die Aktualisierbarkeit der Dokumentation ist differenziert zu bewerten. Positiv ist die hohe Pflegeaktivität. Zahlreiche Seiten wurden im Jahr 2026 überarbeitet, und durch Sprint-Reviews entstehen fortlaufend neue Informationen. Die im Projekt eingesetzte Concurrent Documentation verfolgt außerdem den Ansatz, Dokumentation parallel zur Entwicklung zu erstellen. Dies reduziert das Risiko, dass Dokumentationsaufgaben dauerhaft aufgeschoben werden.

Gleichzeitig enthalten einzelne Dokumente konkrete Servernamen, absolute Netzwerkpfade, Versionsnummern oder Taxonomiebezeichnungen. Solche Angaben können sich kurzfristig ändern und erhöhen den Pflegeaufwand. Auch die Architekturübersicht stellt lediglich eine datierte Momentaufnahme dar. Ohne regelmäßige oder automatisierte Aktualisierung besteht die Gefahr, dass sie nicht mehr mit dem tatsächlichen Stand des Systems übereinstimmt. Unvollständige Platzhalter für Abbildungen zeigen zudem, dass parallel erstellte Dokumentation nicht in jedem Fall vollständig abgeschlossen wird.

Bei Auffindbarkeit und Verständlichkeit bestehen sowohl Stärken als auch Schwächen. Ein zentraler Confluence-Bereich, Inhaltsverzeichnisse, ein Glossar, Jira-Komponenten sowie Verknüpfungen zwischen Jira und dem Repository unterstützen die Suche nach Informationen. Die Dokumentation ist jedoch auf mehrere Bereiche und teilweise persönliche Ablageorte verteilt. Zusätzlich wechseln Sprache und Benennung zwischen Deutsch und Englisch. In einzelnen Fällen tritt dieser Wechsel sogar innerhalb desselben Dokuments auf. Dubletten, uneinheitliche Bezeichnungen und technisch schwer darstellbare Diagramme erschweren damit eine schnelle Orientierung.

Unter Berücksichtigung des Projektkontexts ist der vorhandene Dokumentationsumfang insgesamt angemessen. Finrep ist ein langlebiges, fachlich komplexes und regulatorisch relevantes System mit mehreren Schnittstellen und produktivem Kundeneinsatz. Eine minimale Dokumentation wäre für dieses Projekt daher nicht ausreichend. Die bestehenden Defizite betreffen weniger die Menge als vielmehr die Konsistenz, Aktualität und gezielte Dokumentation besonders komplexer Bereiche.

## 4.3 Handlungsempfehlung

Aus der Bewertung ergibt sich nicht die Notwendigkeit, den Dokumentationsumfang pauschal zu erhöhen. Stattdessen sollte der vorhandene Aufwand gezielter auf Inhalte konzentriert werden, bei denen fehlende oder veraltete Informationen ein hohes Risiko für Entwicklung, Wartung oder Betrieb darstellen.

Zunächst sollte die Dokumentation im Quellcode an fachlich und technisch komplexen Stellen ergänzt werden. Hierzu gehören insbesondere die Berechnungs- und Validierungslogik, zentrale

Geschäftsobjekte sowie Datenzugriffsmethoden mit umfangreichen SQL-Abfragen. Kommentare sollten nicht lediglich die sichtbare Implementierung wiederholen, sondern den Zweck, fachliche Besonderheiten, relevante Parameter und betroffene Datenstrukturen erläutern. Eine flächendeckende Kommentierung sämtlicher Klassen und Methoden ist dagegen nicht erforderlich.

Dokumente, die aus dem aktuellen Systemzustand abgeleitet werden können, sollten möglichst automatisiert erzeugt werden. Dies betrifft insbesondere die Architekturübersicht. Eine Einbindung der Generierung in die bestehende Continuous-Integration-Pipeline könnte sicherstellen, dass Architekturinformationen regelmäßig aktualisiert und mit einem Erstellungsdatum sowie dem zugehörigen Quellcode-Stand versehen werden. Veränderliche Angaben wie Servernamen, Netzwerkpfade oder Versionsbezeichnungen sollten außerdem nicht mehrfach in Fließtexten gepflegt, sondern über zentrale Konfigurationen oder Verweise eingebunden werden.

Zur Verbesserung der Auffindbarkeit sollte eine zentrale Einstiegsseite geschaffen werden, von der aus die wesentlichen Dokumentationsbereiche erreichbar sind. Inhaltlich ähnliche Seiten sollten zusammengeführt und nicht mehr verwendete Dokumente archiviert werden. Zusätzlich sollte für unterschiedliche Dokumenttypen eine einheitliche Sprache festgelegt werden. Beispielsweise könnten technische Dokumentationen überwiegend auf Englisch und kunden- oder supportbezogene Inhalte auf Deutsch geführt werden. Entscheidend ist dabei weniger die gewählte Sprache als deren konsequente Verwendung.

Die Dokumentationspflege sollte stärker organisatorisch in den Entwicklungsprozess eingebunden werden. Bei Änderungen an dokumentierten Funktionen sollte bereits innerhalb der jeweiligen Story oder Fehlerbehebung geprüft werden, ob auch README-Dateien, Confluence-Seiten oder Betriebsanleitungen angepasst werden müssen. Eine entsprechende Prüfung kann als Bestandteil der Definition of Done festgelegt werden. Für zentrale Dokumente sollten außerdem verantwortliche Personen sowie regelmäßige Überprüfungs- und Archivierungsintervalle bestimmt werden.

Technische Entscheidungen mit langfristigen Auswirkungen sollten durch kurze Architecture Decision Records festgehalten werden. Dadurch können nicht nur die getroffenen Entscheidungen, sondern auch ihre Ausgangssituation, Begründung und mögliche Alternativen nachvollzogen werden. Dies ist insbesondere bei Taxonomieumstellungen, der Auswahl von Berechnungsmechanismen oder Änderungen an externen Schnittstellen sinnvoll.

Bei begrenzten zeitlichen Ressourcen sollte die Dokumentation risikobasiert priorisiert werden. Vorrangig zu behandeln sind die Zentralbank-Schnittstellen, die Taxonomie- und Importprozesse sowie die Berechnungs- und Validierungslogik. Diese Bereiche sind für den Betrieb besonders kritisch und zugleich nur mit hohem Aufwand aus dem Quellcode rekonstruierbar.

Insgesamt benötigt Finrep daher nicht grundsätzlich mehr Dokumentation, sondern eine konsistentere, leichter auffindbare und dauerhaft aktualisierbare Dokumentationsbasis. Durch gezielte In-Code-Dokumentation, automatisierte Aktualisierung, klare Verantwortlichkeiten und eine risikobasierte Priorisierung kann das vorhandene Projektwissen besser gesichert werden, ohne den agilen Entwicklungsprozess durch unverhältnismäßigen Dokumentationsaufwand zu belasten.

## 5 Fazit

### 5.1 Beantwortung der Forschungsfrage

In agilen Softwareprojekten ist so viel Dokumentation sinnvoll, wie zur Sicherung von Projektwissen sowie zur Unterstützung von Entwicklung, Wartung und Betrieb erforderlich ist. Unnötige, redundante oder nicht dauerhaft pflegbare Inhalte sollten dagegen vermieden werden. Ein fester zeitlicher Anteil ist daher nicht zweckmäßig; der Dokumentationsaufwand sollte abhängig von Komplexität, Kritikalität, Lebensdauer und externen Abhängigkeiten festgelegt werden.

Für Finrep ist der bestehende Dokumentationsumfang aufgrund der hohen fachlichen und technischen Komplexität grundsätzlich angemessen. Verbesserungsbedarf besteht weniger bei der Menge als bei der Qualität, insbesondere hinsichtlich der In-Code-Dokumentation, der langfristigen Aktualität sowie einer einheitlichen Struktur und Sprache.

### 5.2 Ausblick

Zukünftig sollten die empfohlenen Maßnahmen schrittweise in den Entwicklungsprozess integriert werden. Dazu gehören insbesondere die gezielte Dokumentation kritischer Codebereiche, die organisatorische Verankerung der Dokumentationspflege und die automatische Aktualisierung technischer Übersichten. Anschließend kann überprüft werden, ob sich dadurch beispielsweise Einarbeitung, Wartung und Fehleranalyse messbar verbessern.

# Bibliographie

- [1] P. Abrahamsson, O. Salo, J. Ronkainen, und J. Warsta, „Agile Software Development Methods: Review and Analysis“, *Proc. Espoo 2002*, S. 3–107, Jan. 2002, doi: <https://doi.org/10.48550/arXiv.1709.08439>.
- [2] T. Dybå und T. Dingsøyr, „Empirical studies of agile software development: A systematic review“, *Information and Software Technology*, Bd. 50, Nr. 9, S. 833–859, 2008, doi: <https://doi.org/10.1016/j.infsof.2008.01.006>.
- [3] K. Conboy, „Agility from First Principles: Reconstructing the Concept of Agility in Information Systems Development“, *Information Systems Research*, Bd. 20, Nr. 3, S. 329–354, 2009, doi: [10.1287/isre.1090.0236](https://doi.org/10.1287/isre.1090.0236).
- [4] N. B. Moe, T. Dingsøyr, und T. Dybå, „A teamwork model for understanding an agile team: A case study of a Scrum project“, *Information and Software Technology*, Bd. 52, Nr. 5, S. 480–491, 2010, doi: <https://doi.org/10.1016/j.infsof.2009.11.004>.
- [5] B. Boehm und R. Turner, „Management challenges to implementing agile processes in traditional development organizations“, *IEEE Software*, Bd. 22, Nr. 5, S. 30–39, 2005, doi: <https://doi.org/10.1109/MS.2005.129>.
- [6] T. Lethbridge, J. Singer, und A. Forward, „How software engineers use documentation: the state of the practice“, *IEEE Software*, Bd. 20, Nr. 6, S. 35–39, 2003, doi: <https://doi.org/10.1109/MS.2003.1241364>.
- [7] A. Forward und T. C. Lethbridge, „The relevance of software documentation, tools and technologies: a survey“, in *Proceedings of the 2002 ACM Symposium on Document Engineering*, in DocEng '02. McLean, Virginia, USA: Association for Computing Machinery, 2002, S. 26–33. doi: <https://doi.org/10.1145/585058.585065>.
- [8] C. Treude, J. Middleton, und T. Atapattu, „Beyond accuracy: assessing software documentation quality“, in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, in ESEC/FSE 2020. Virtual Event, USA: Association for Computing Machinery, 2020, S. 1509–1512. doi: <https://doi.org/10.1145/3368089.3417045>.
- [9] F. Paetsch, A. Eberlein, und F. Maurer, „Requirements engineering and agile software development“, in *WET ICE 2003. Proceedings. Twelfth IEEE International Workshops on Enabling Technologies: Infrastructure for Collaborative Enterprises, 2003.*, 2003, S. 308–313. doi: <https://doi.org/10.1109/ENABL.2003.1231428>.
- [10] V. T. Heikkilä, D. Damian, C. Lassenius, und M. Paasivaara, „A Mapping Study on Requirements Engineering in Agile Software Development“, in *2015 41st Euromicro Conference on Software Engineering and Advanced Applications*, 2015, S. 199–207. doi: <https://doi.org/10.1109/SEAA.2015.70>.
- [11] S. C. B. de Souza, N. Anquetil, und K. M. de Oliveira, „A study of the documentation essential to software maintenance“, in *Proceedings of the 23rd Annual International Conference on Design of Communication: Documenting & Designing for Pervasive Information*, in SIGDOC '05. Coventry, United Kingdom: Association for Computing Machinery, 2005, S. 68–75. doi: <https://doi.org/10.1145/1085313.1085331>.
- [12] C. J. Stettina und W. Heijstek, „Necessary and neglected? an empirical study of internal documentation in agile software development teams“, in *Proceedings of the 29th ACM International Conference on Design of Communication*, in SIGDOC '11. Pisa, Italy: Association for Computing Machinery, 2011, S. 159–166. doi: <https://doi.org/10.1145/2038476.2038509>.
- [13] E. Bjarnason, K. Wnuk, und B. Regnell, „Requirements are slipping through the gaps — A case study on causes & effects of communication gaps in large-scale software development“, in *2011 IEEE 19th*

- International Requirements Engineering Conference*, 2011, S. 37–46. doi: <https://doi.org/10.1109/RE.2011.6051639>.
- [14] B. Ramesh, L. Cao, und R. Baskerville, „Agile requirements engineering practices and challenges: an empirical study“, *Information Systems Journal*, Bd. 20, Nr. 5, S. 449–480, 2010, doi: <https://doi.org/10.1111/j.1365-2575.2007.00259.x>.
- [15] I. Inayat, S. S. Salim, S. Marczak, M. Daneva, und S. Shamshirband, „A systematic literature review on agile requirements engineering practices and challenges“, *Computers in Human Behavior*, Bd. 51, S. 915–929, 2015, doi: <https://doi.org/10.1016/j.chb.2014.10.046>.
- [16] R. Kasauli, E. Knauss, J. Horkoff, G. Liebel, und F. G. de Oliveira Neto, „Requirements engineering challenges and practices in large-scale agile system development“, *Journal of Systems and Software*, Bd. 172, S. 110851, 2021, doi: <https://doi.org/10.1016/j.jss.2020.110851>.
- [17] M. A. Islam, R. Hasan, und N. U. Eisty, „Documentation Practices in Agile Software Development: A Systematic Literature Review“, in *2023 IEEE/ACIS 21st International Conference on Software Engineering Research, Management and Applications (SERA)*, 2023, S. 266–273. doi: <https://doi.org/10.1109/SERA57763.2023.10197828>.